

Lecture 29 Heaps

Chapter 12 of textbook

1. Concept of heaps
2. Binary heaps
3. Binomial heaps
4. Fibonacci heaps
5. Heap summary

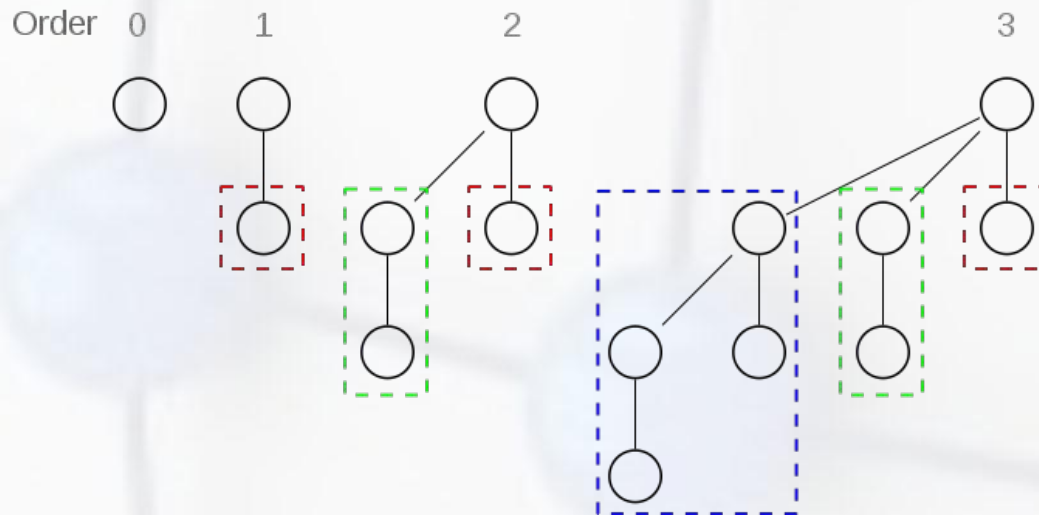
2. Binomial Heap

Binomial Heap is defined by Binomial tree.

Binomial tree is an ordered tree that can be recursively defined as:

1. A binomial tree of order 0 has a single node.
2. A binomial tree of order i has a root node whose children are root nodes of binomial trees of order $i-1, i-2, \dots, 2, 1, 0$.

Example



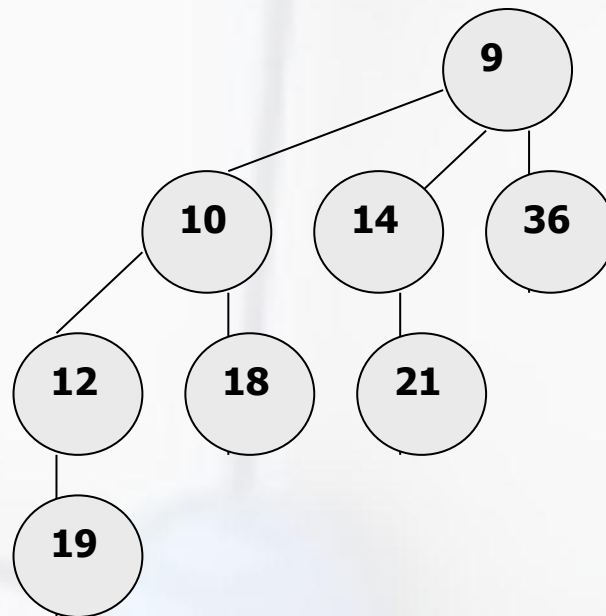
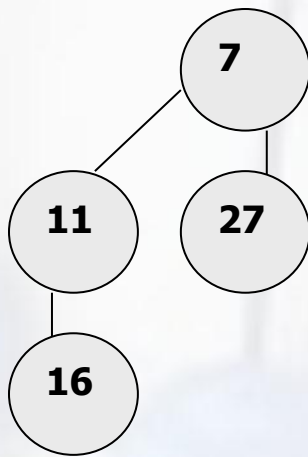
Properties of Binomial Tree

1. A binomial tree order i , denoted B_i , has 2^i nodes
2. The height of a binomial tree B_i is i
3. The root has i children
4. A binomial tree B_i consists of two binomial trees B_{i-1} linked together such that root of one is the leftmost child of the root of the other
5. There are $C(i, h)$ node at level h

Binomial Heap

A Binomial Heap is a set of Binomial Trees with two properties:

- each binomial tree follows min-heap property
- there can be at-most one binomial tree of any degree.



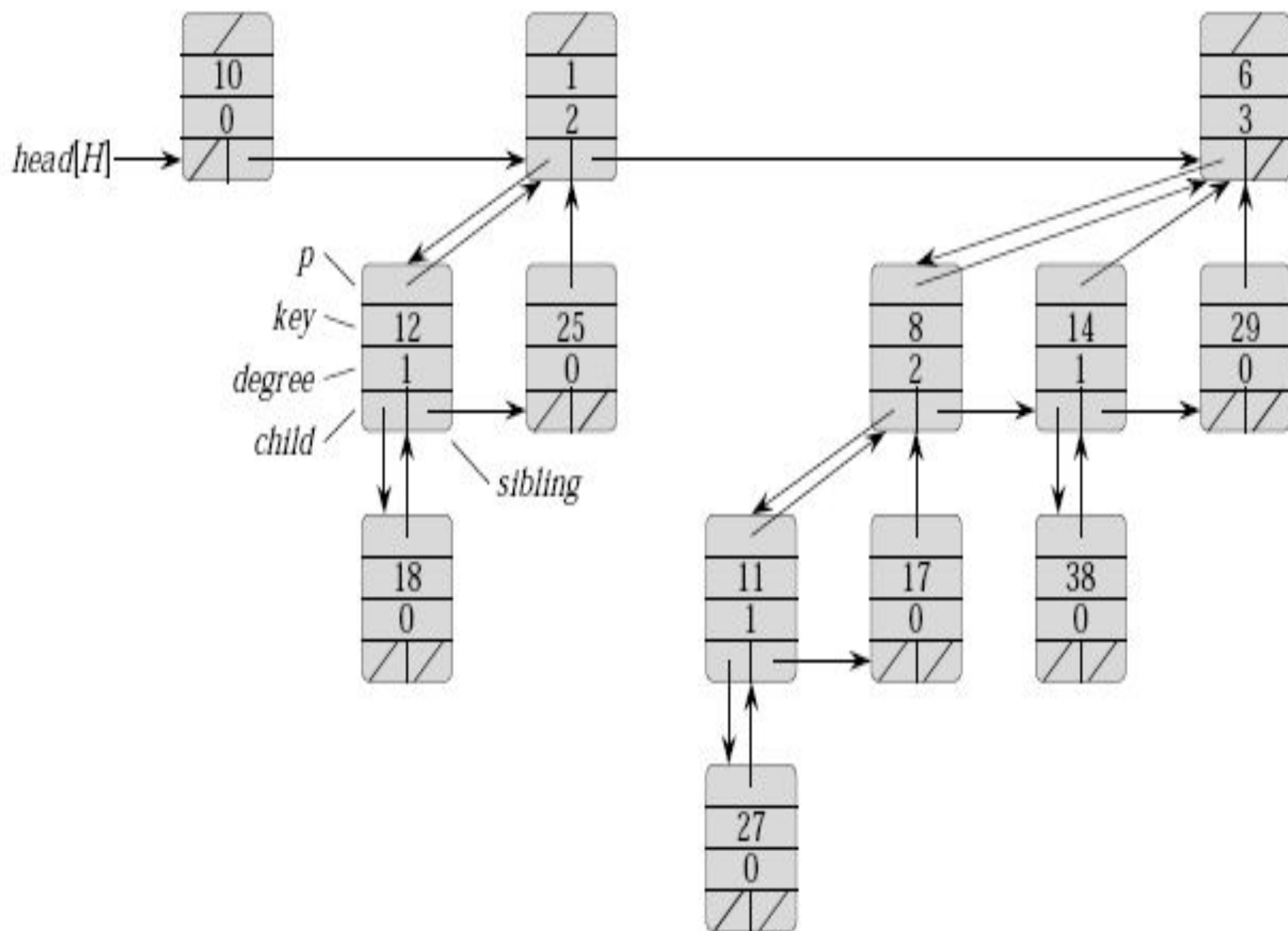
Binomial Heap

Derived properties:

- root of each binomial tree contains the smallest key in the tree
- a binomial heap H of N nodes contains at most $\log N + 1$ binomial trees

Linked node representation of Binomial Heaps

- Each node N has following fields
 - a field that stores its key
 - a field d stores the degree of N
 - pointer $P[N]$ points to the parent of N
 - *pointer Child* $[N]$ points to the leftmost child
 - *Pointer Sibling* $[N]$ that point to the right sibling of N
- If N is the root node, then $P[N] = \text{NULL}$. If N has no children, then $\text{Child}[N] = \text{NULL}$, and if N is the rightmost child of its parent, then $\text{sibling}[N] = \text{NIL}$.
- The root nodes of the binomial trees are organized in a linked list with increasing order of degree, called ***root list***.



Operations on Binomial Heaps

- find-min
- delete-min
- Union
- Insert
- Dec-key

Operations on Binomial Heaps

- Find-min

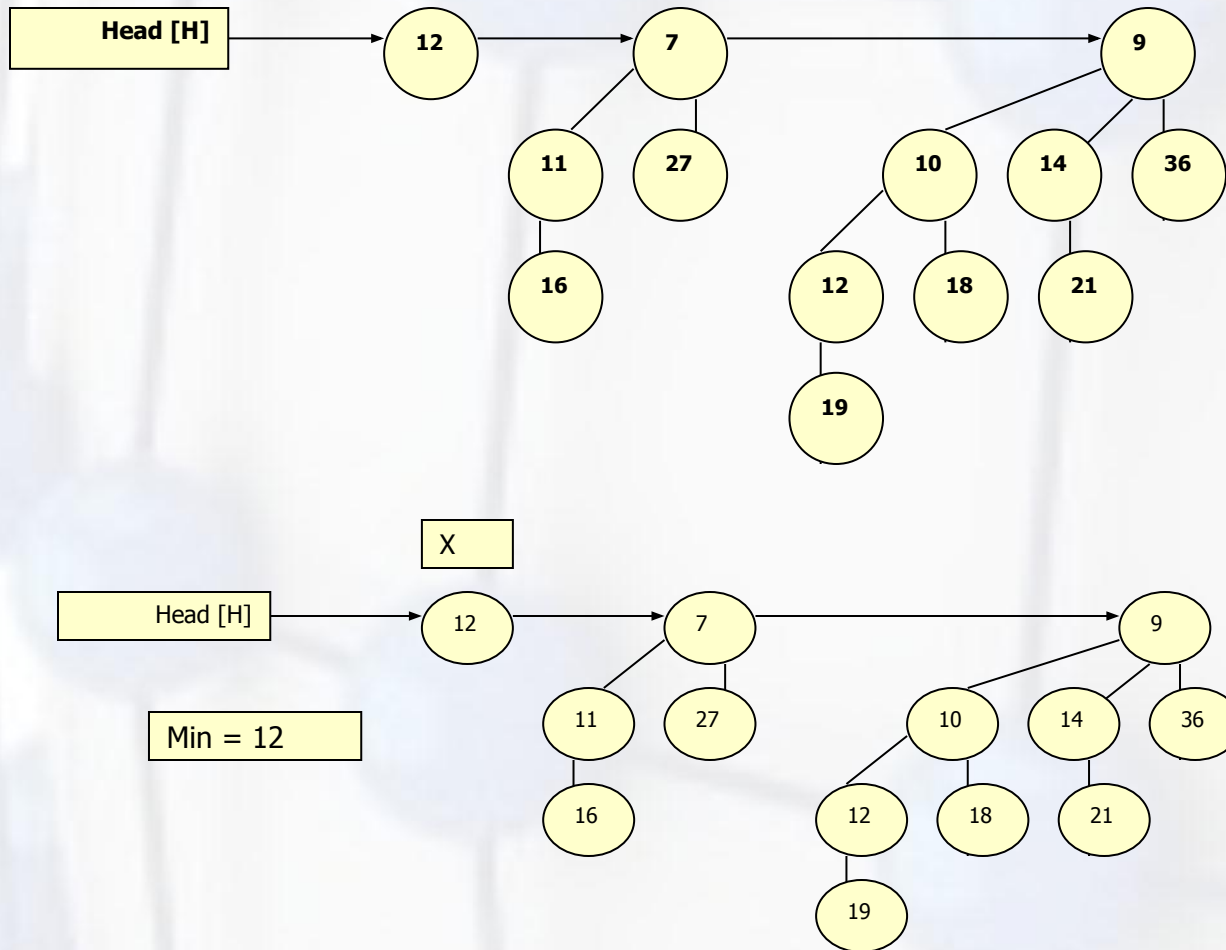
The procedure returns a pointer to the node having minimum value in the binomial heap H .

A binomial heap is heap-ordered, the node with minimum value in a particular binomial tree is at the root node of the binomial tree. The procedure checks all roots.

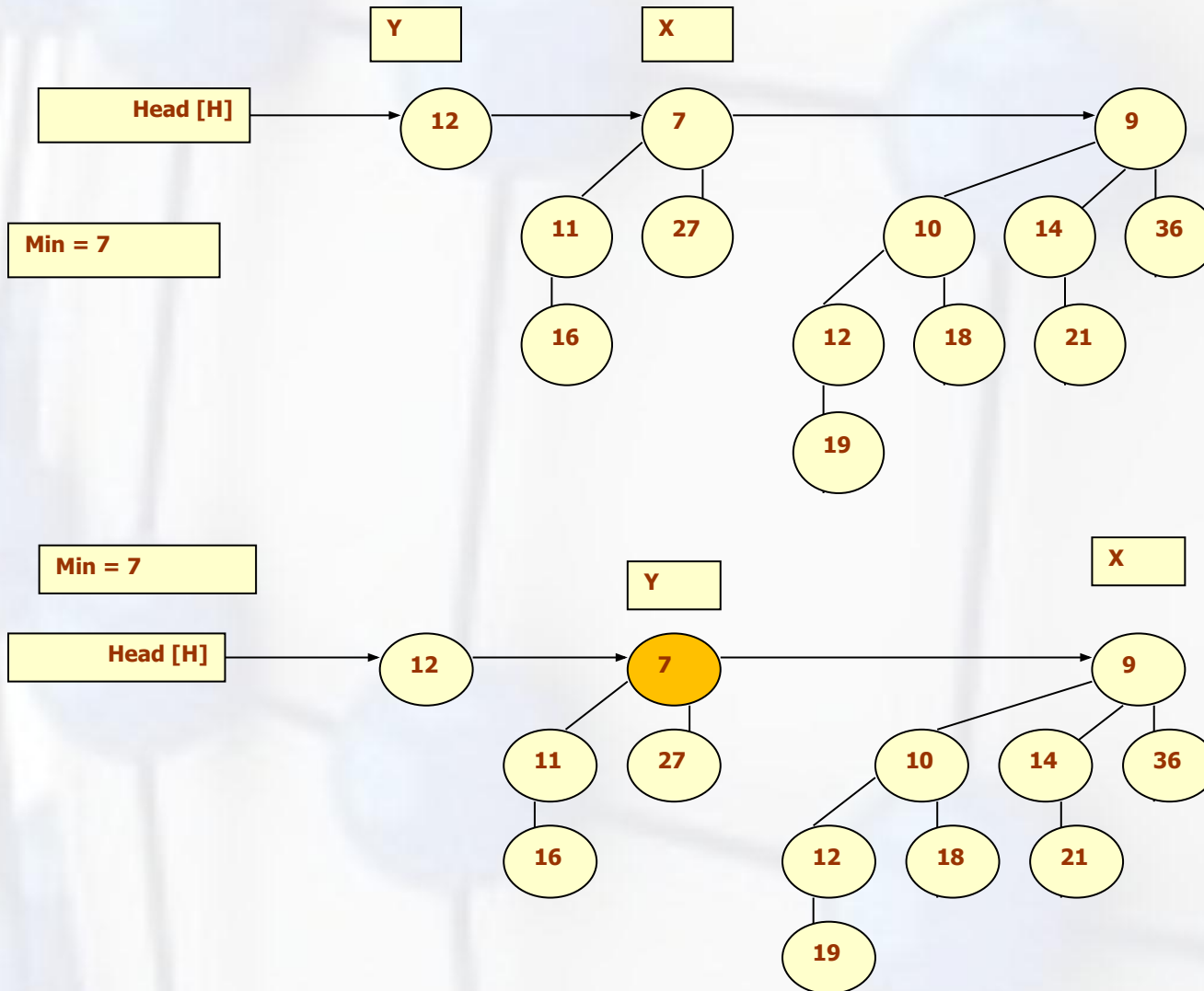
Since there are at most $\log n + 1$ roots to check, the running time of this procedure is $O(\log n)$.

Operations on Binomial Heaps

Example: find-min



Operations on Binomial Heaps



Union two Binomial Heaps

- Union -- the procedure of uniting two binomial heaps into one binomial heap

Algorithm: given binomial heaps H1 and H2

Step 1. merge H1 and H2, i.e. link the roots of H1 and H2 in non-decreasing order.

Step 2. restoring binomial heap by linking binomial trees of the same

degree together: traverse the linked list, keep track of three pointers, prev, ptr and next.

Case 1: degrees of ptr and next are not same, move ahead.

Case 2: If degree of next->next is also same, move ahead.

Case 3: If key of x is smaller than or equal to key of next, make next as a child of ptr by linking it with ptr.

Case 4: If key of ptr is greater, then make ptr as child of

Uniting two Binomial Heaps

Union_Binomial-Heap(H_1, H_2)

Step 1: SET $H = \text{Create_Binomial-Heap}()$

Step 2: SET $\text{Head}[H] = \text{Merge_Binomial-Heap}()$

Step 3: Free the memory occupied by H_1 and H_2

Step 4: If $\text{Head}[H] = \text{NULL}$, then RETURN

Step 5: SET $\text{PREV} = \text{NULL}$, $\text{PTR} = \text{Head}[H]$ and $\text{NEXT} = \text{Sibling}[\text{PTR}]$

Step 6: Repeat Step 7 while $\text{NEXT} \neq \text{NULL}$

Step 7: IF $\text{Degree}[\text{PTR}] \neq \text{Degree}[\text{NEXT}]$ OR $(\text{Sibling}[\text{NEXT}] \neq \text{NULL}$
AND $\text{Degree}[\text{Sibling}[\text{NEXT}] = \text{Degree}[\text{PTR}])$, then

 SET $\text{PREV} = \text{PTR}$, $\text{PTR} = \text{NEXT}$

 ELSE IF $\text{Val}[\text{PTR}] \leq \text{Val}[\text{NEXT}]$, then

 SET $\text{Sibling}[\text{PTR}] = \text{Sibling}[\text{NEXT}]$

 Link_Binomial-Tree(NEXT , PTR)

 Else

 IF $\text{PREV} = \text{NULL}$, then

$\text{Head}[H] = \text{NEXT}$

 ELSE

$\text{Sibling}[\text{PREV}] = \text{NEXT}$

 Link_Binomial-Tree(PTR , NEXT)

 SET $\text{PTR} = \text{NEXT}$

 SET $\text{NEXT} = \text{Sibling}[\text{PTR}]$

Step 8: RETURN H

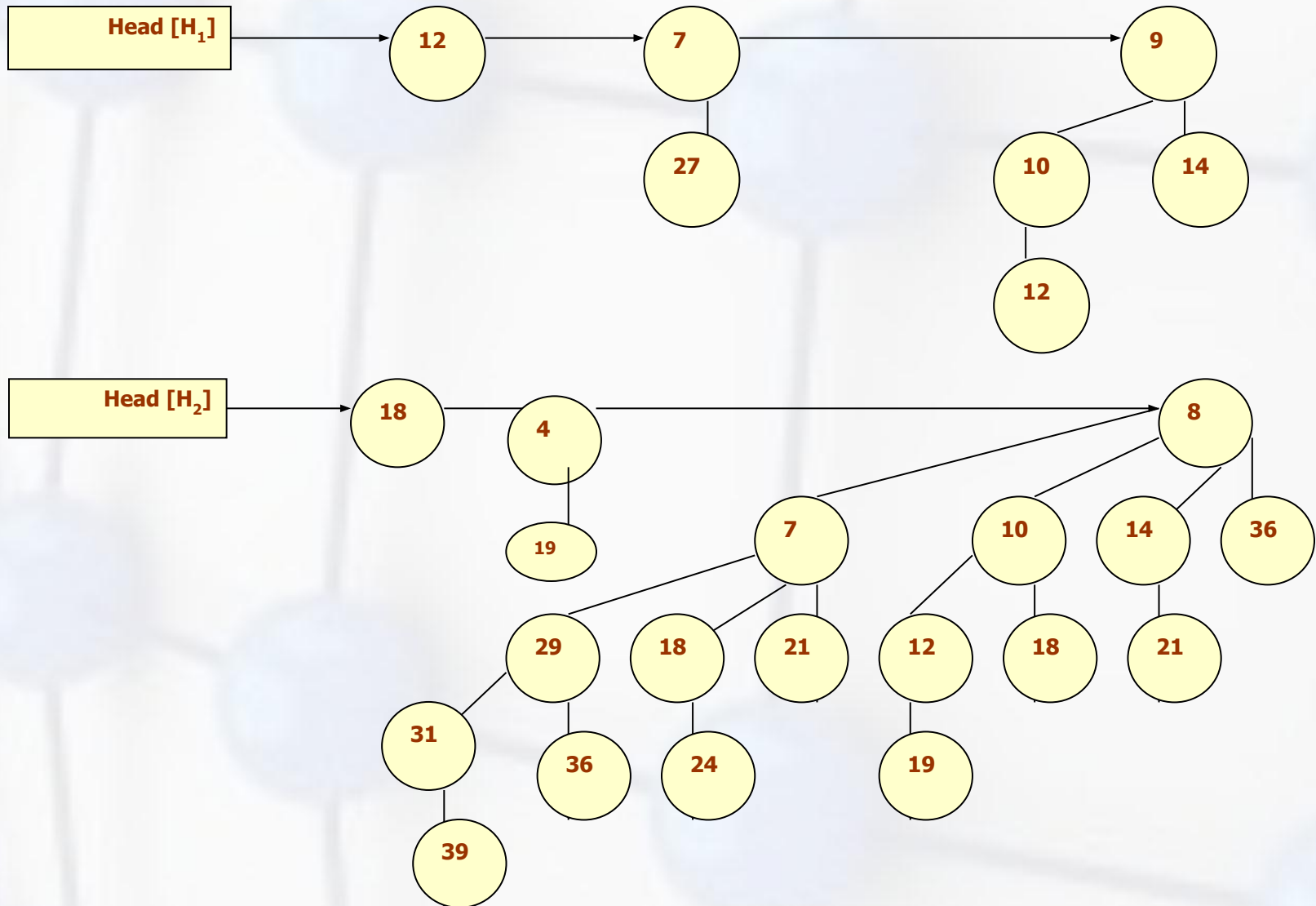
Uniting two Binomial Heaps

Time complexity of union operation

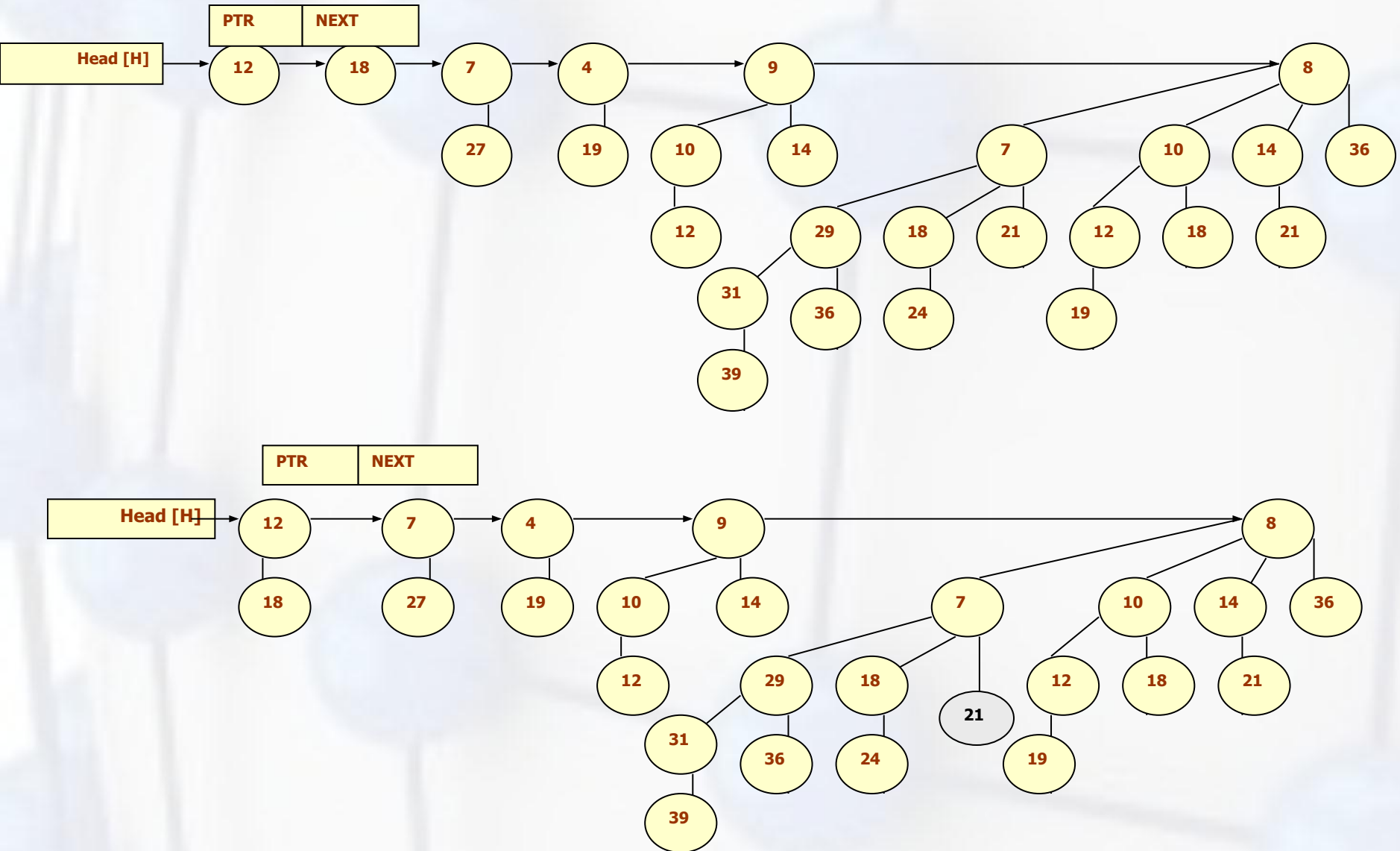
- If H_1 contain n_1 nodes and H_2 contain n_2 nodes, then H_1 contains at most $\log n_1 + 1$ roots and H_2 contains at most $\log n_2 + 1$ roots, H contains at most $\log n_2 + \log n_1 + 2 \leq 2 \log n + 2 = O(\log n)$
- Since, $n = n_1 + n_2$, the merge takes $O(\log n)$ time.

Each iteration (the step 2) takes $O(1)$ time. There are at most $\log n_1 + \log n_2 + 2$ iterations, the total time is thus $O(\log n)$.

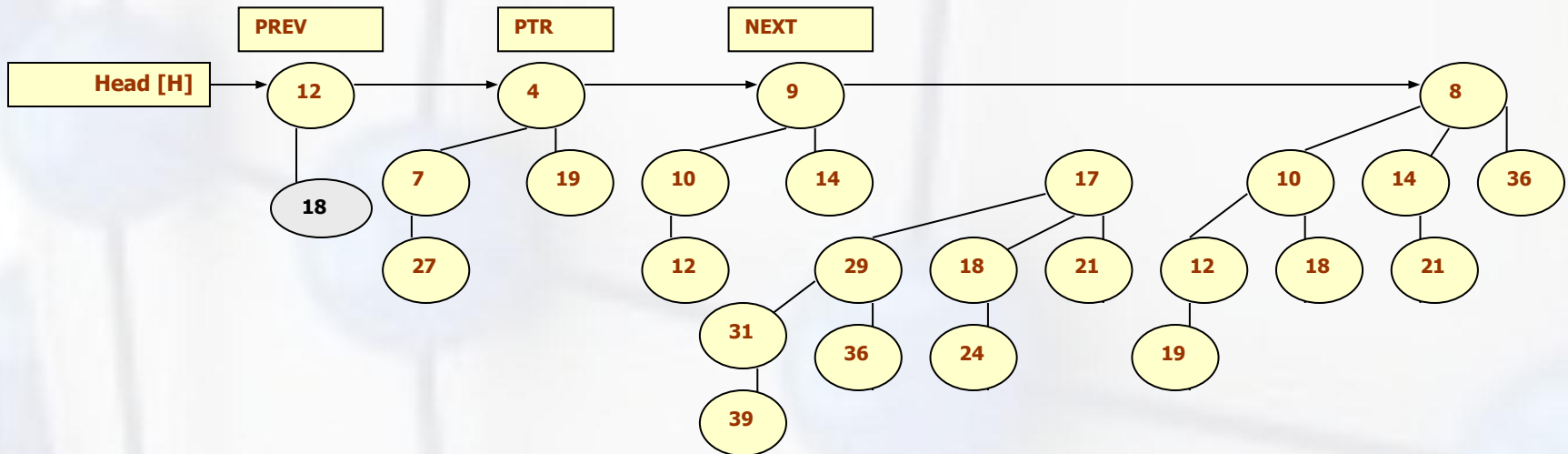
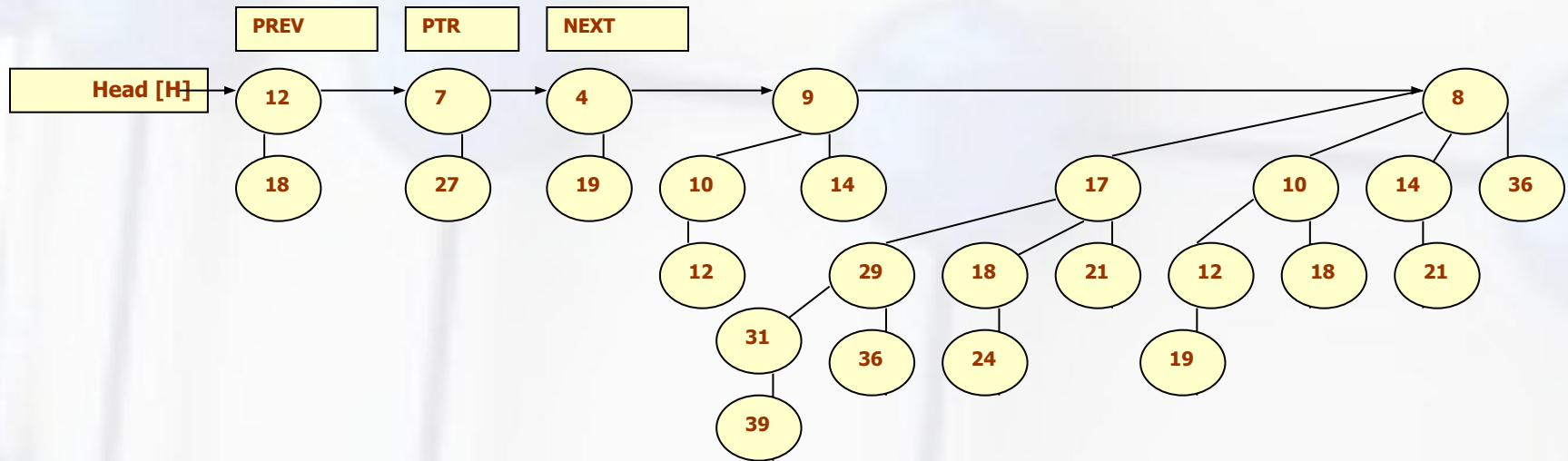
Example of Union operation



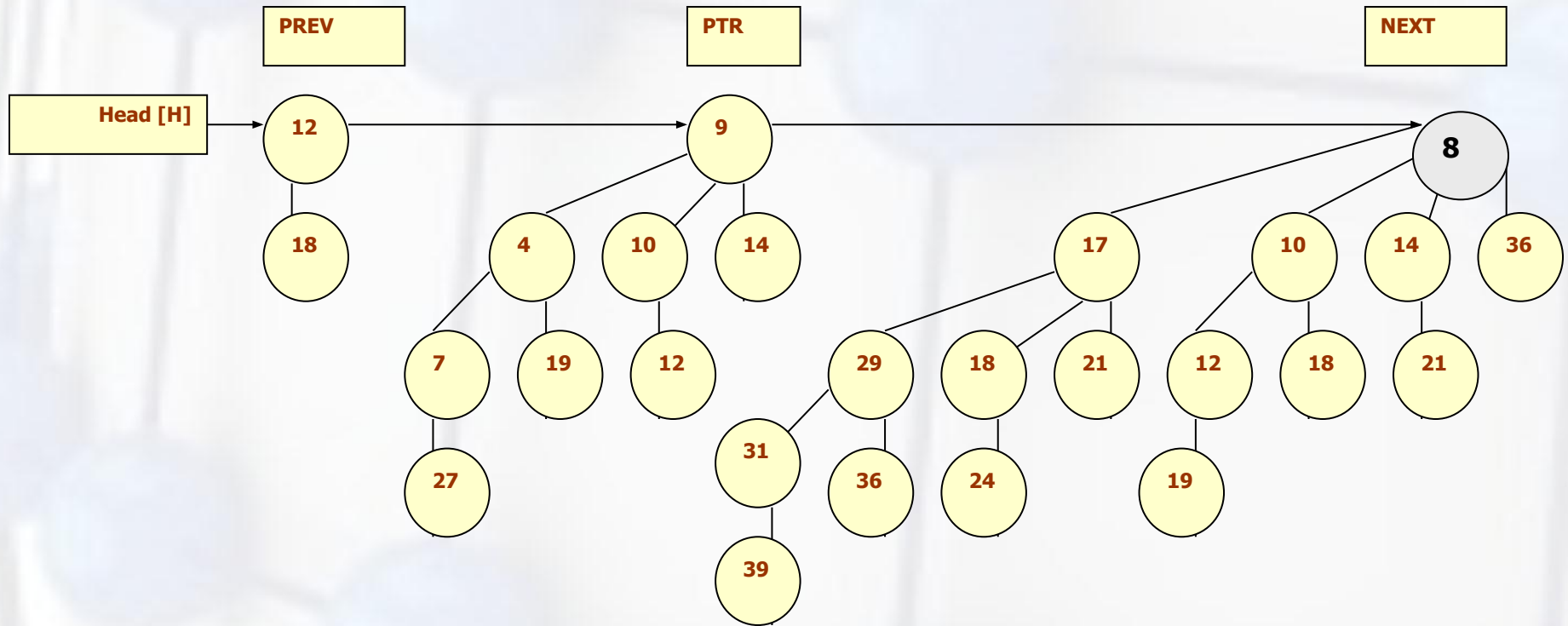
Example of union operation



Example of union operation



Unite the Binomial Heaps given below



Inserting a new node

Insert --- insert a node x into binomial heap H .

Algorithm

Step 1. create a new binomial heap H_1 of one node of value x

Step 2. Unite H_1 and H .

`Insert_Binomial_Heap()`

Step 1: SET $H' = \text{Create_Binomial_Heap}()$

Step 2: SET $\text{Parent}[x] = \text{NULL}$, $\text{Child}[x] = \text{NULL}$ and $\text{sibling}[x] = \text{NULL}$,
 $\text{Degree}[x] = \text{NULL}$

Step 3: SET $\text{Head}[H'] = x$

Step 4: SET $\text{Head}[H] = \text{Union_Binomial_Heap}(H, H')$

Step 5: END

Time complexity: ? Space complexity: ?

extracting-min / delete-min

- Delete-min – delete node with minimum key from binomial heap H.

Algorithm

Step 1. call find-min to find min node x of H

Step 2. remove x from root list of H

Step 3. create a new binomial heap H1 by delete x and reverse its

children

Step 4. call union(H, H1)

Time complexity: ? Space complexity: ?

Extracting the node with minimum key

`Min-Extract_Binomial Heap (H)`

Step 1: Find the root *R* having minimum value in the root list of *H*

Step 2: Remove *R* from the root list of *H*

Step 3: SET *H'* = `Create_Binomial-Heap()`

Step 4: Reverse the order of *R*'s children thereby forming a linked list

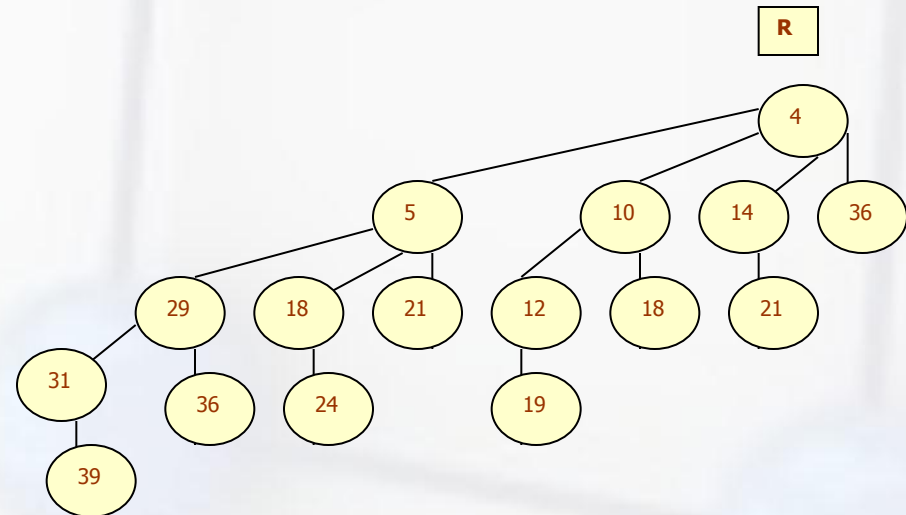
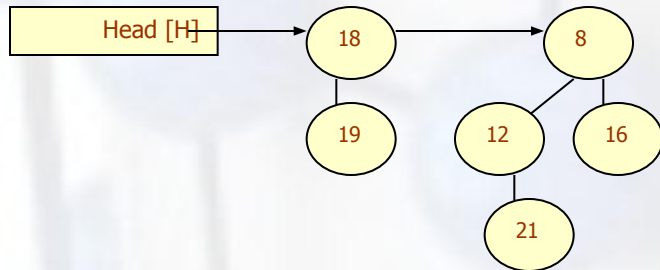
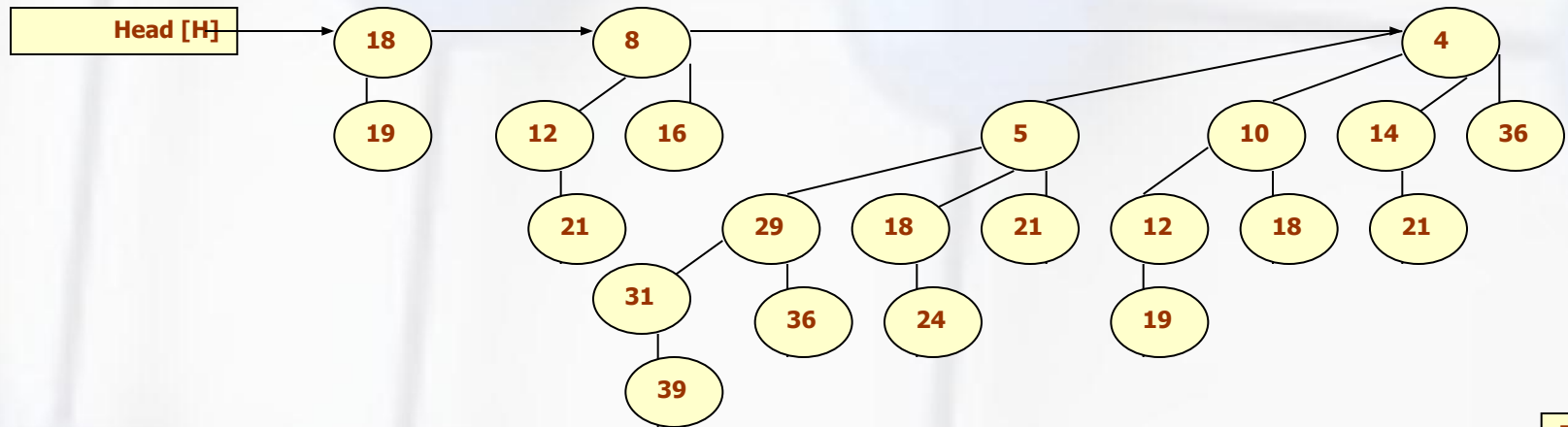
Step 5: Set *head*[*H'*] to point to the head of the resulting list

Step 6: SET *H* = `Union_Binomial-Heap(H, H')`

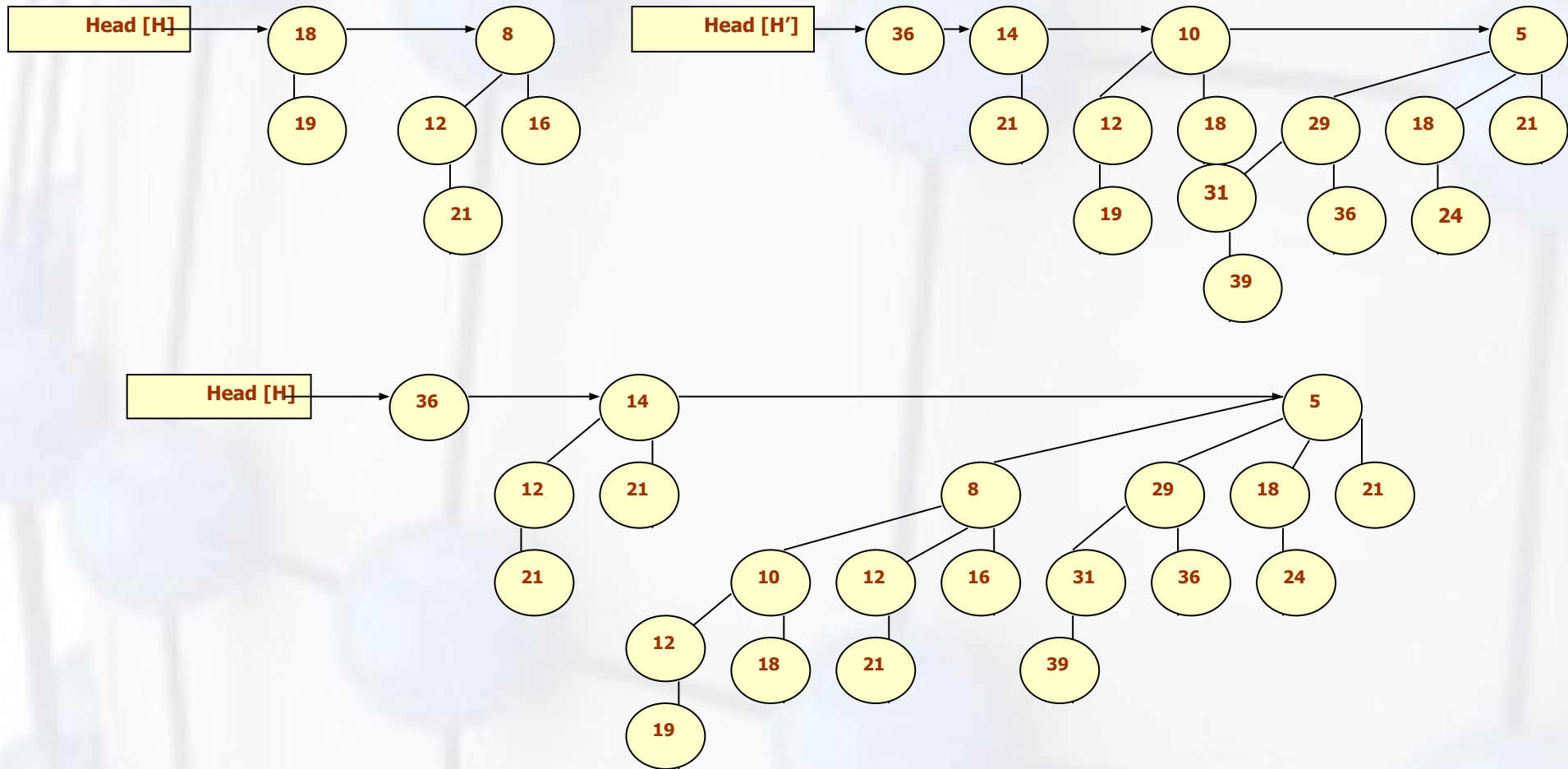
Step 7: Return *R*

Extracting the node with minimum key

Example: Extract the node with minimum value from the binary heap given below.



Extracting the node with minimum key



Decreasing a value of a node

- The algorithm to decrease the value of a node x in a binomial heap H is given below. In the algorithm the value of the node is overwritten with a new value k , less than the current value of the node. The Binomial-Heap_Decrease_Val procedure takes $O(\lg n)$ time as the maximum depth of node x is $\lg n$, so the while loop will iterate at most $\lg n$ times.

Binomial-Heap_Decrease_Val(H, x, k)

Step 1: IF Val[x] < k , then Print " ERROR"

Step 2: SET Val[x] = k

Step 3: SET PTR = x and PAR = Parent[PTR]

Step 4: Repeat while PAR $\neq$ NULL and Val[PTR] < Val[PAR]

Step 5: SWAP (Val[PTR], Val[PAR])

Step 6:

Deleting a node

- Once we have understood the Binomial-Heap_Decrease_Val procedure, it becomes very easy to delete a node x 's value from binomial heap H in $O(\lg n)$ time. To start with the algorithm we set the value of x to $-\infty$. Assuming that there is no node in the heap that has a value less than $-\infty$. The algorithm to delete a node from a binomial heap can be given as below.

Binomial-Heap_Delete-Node(H, x)

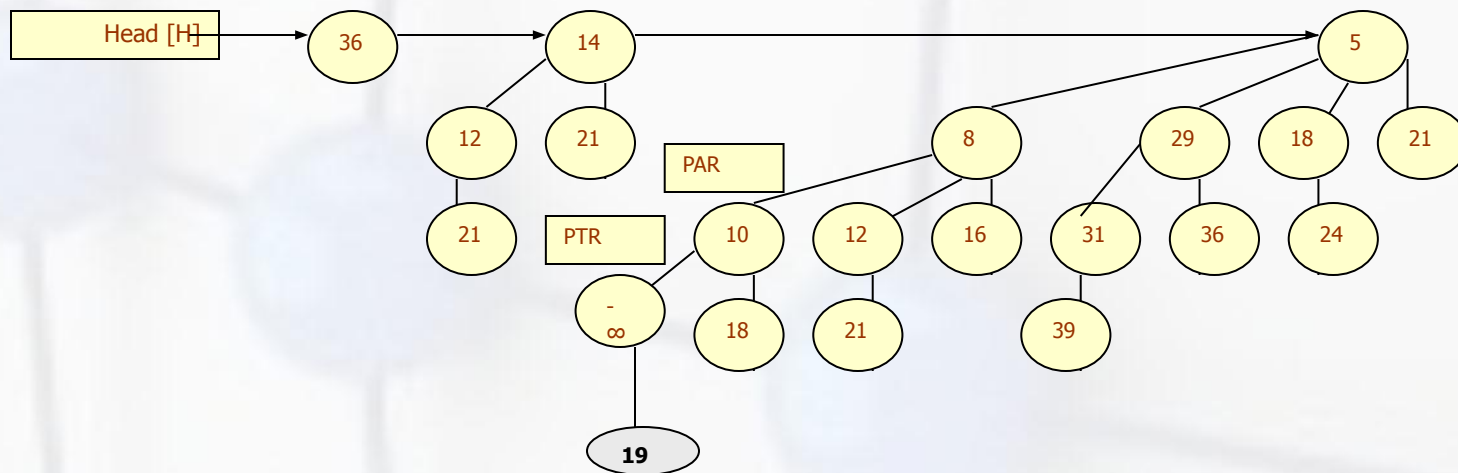
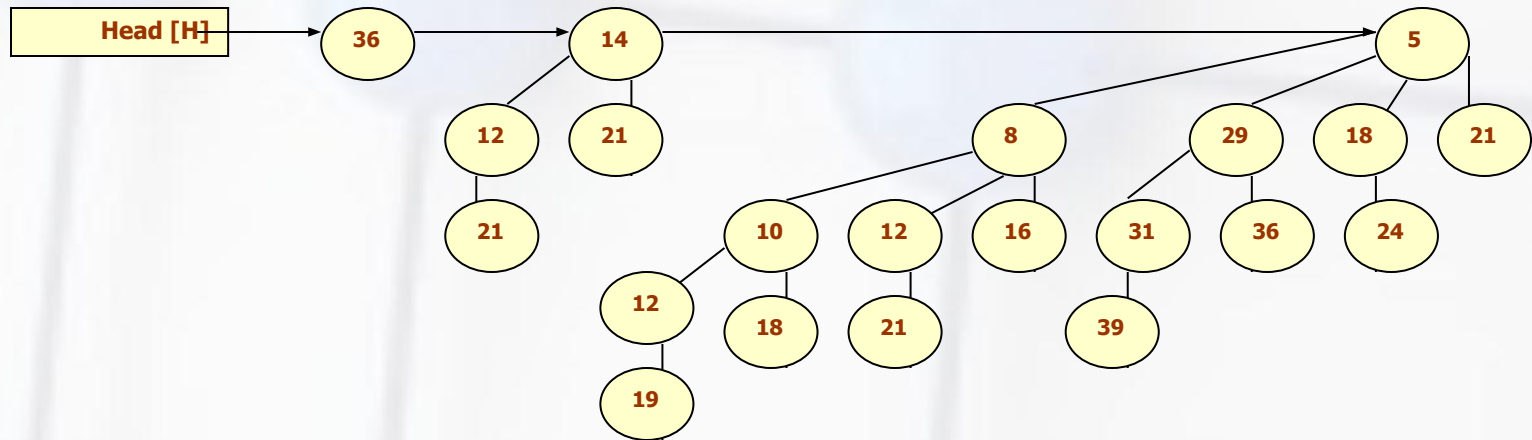
Step 1: Binomial-Heap_Decrease_Val($H, x, -\infty$)

Step 2: Min-Extract_Binomial-Heap(H)

Step 3: END

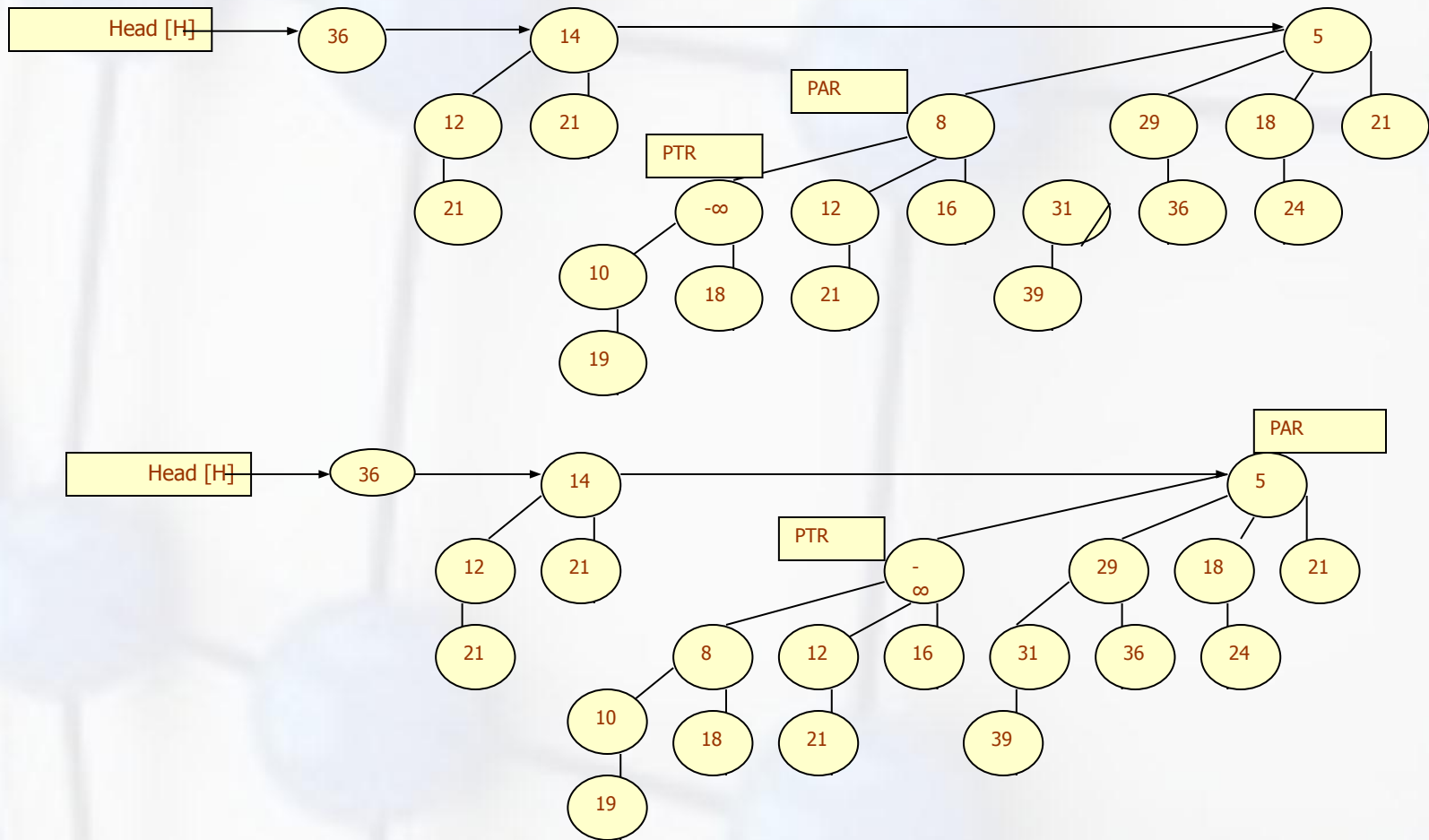
Deleting a key

Example: Delete the node with value 12 from the binomial heap H.



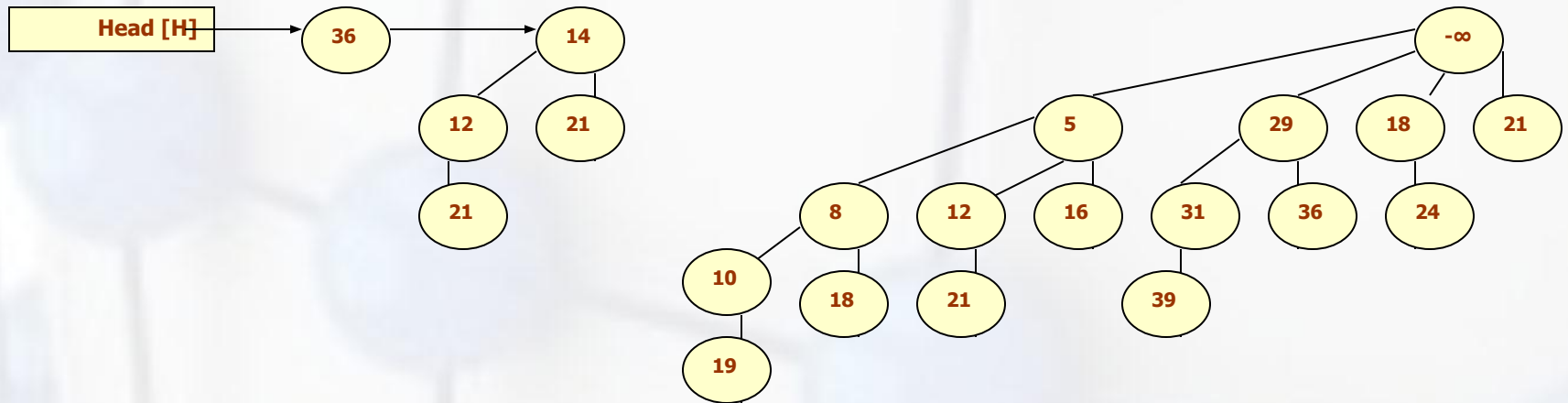
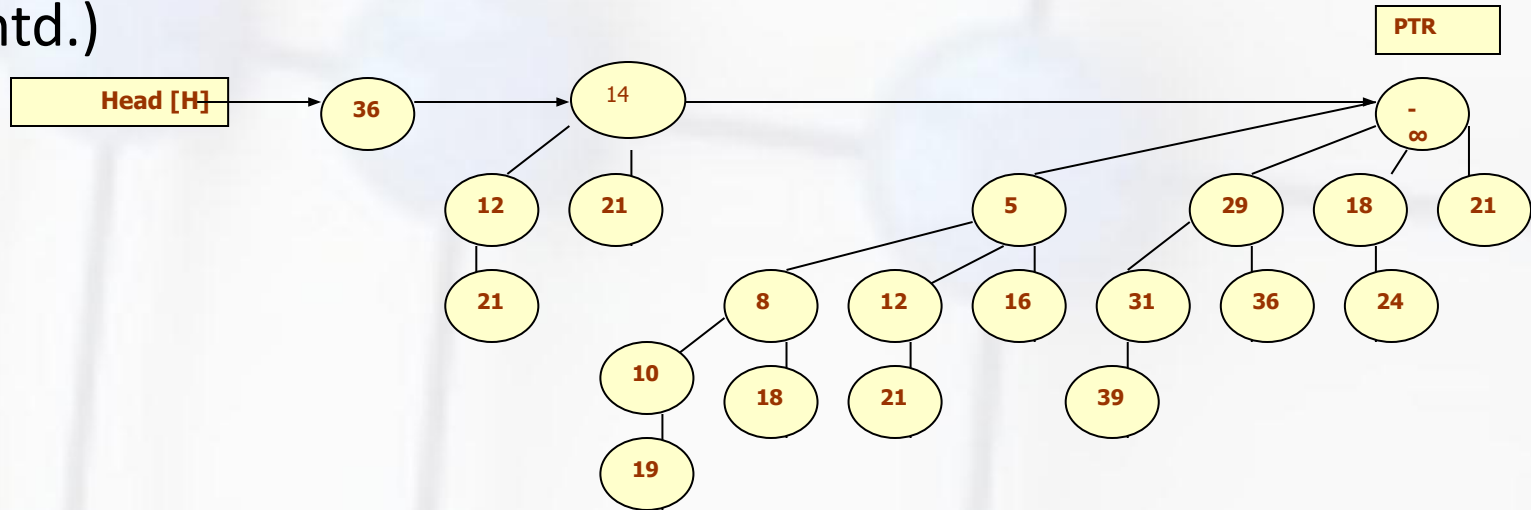
Deleting a key

(Contd.)



Deleting a key

(Contd.)



Deleting a key

(Contd.)

